

MARL Meets AlphaChip: Thermal-Aware 3D Floorplanning for Heterogeneous Multi-Die SoCs

Abstract—Modern 3D heterogeneous systems demand thermally and spatially optimized floorplanning strategies that go beyond traditional monolithic IC assumptions. The growing compute density of advanced AI chips, particularly in LLM data centers, has led to severe thermal challenges, including real-world failures such as GPU node throttling and partial system outages due to overheating. As a result, liquid cooling is rapidly becoming essential in commercial deployments to mitigate thermal bottlenecks. Existing reinforcement learning-based placement frameworks, such as Google’s AlphaChip, operate effectively in flat 2D environments but fail to scale to chiplet architectures, which require coordinated optimization across vertically stacked dies with complex interconnect and heat dissipation dynamics.

This paper presents a unified multi-agent reinforcement learning (MARL) framework for thermal-aware floorplanning in 3D chiplet-based systems. Each horizontal agent optimizes macro placement within a single tier, while a global vertical agent coordinates cross-tier actions, including macro migration, TSV assignment and dynamic integration of microchannel-based liquid cooling. We integrated a compact RC-network based thermal model with fluid-aware convection primitives to enable agents to reason about spatio-temporal thermal behavior during training. To support scalable training, we designed a custom hardware accelerator with APU cores for parallel policy updates and a dedicated L1 cache for prioritized experience replay. Evaluated on multi-tier industrial-scale benchmarks, our method reduces peak temperature by up to 12.5%, lowers inter-die wirelength by 17.8% and converges 3.2 times faster than GPU-based baselines. These results demonstrate the viability of intelligent MARL floorplanning for future heterogeneous systems with tight thermal constraints and real-time liquid cooling integration.

Keywords— 3D floorplanning, multi-die systems, heterogeneous integration, thermal-aware design, MARL accelerator.

I. INTRODUCTION

The exponential growth of computational demands fueled by large-scale AI models like ChatGPT and an explosion in data-centric applications has exposed fundamental limitations in traditional System-on-Chip (SoC) design. Hyperscale data centers, which house thousands of high-performance GPUs for foundation model inference are now confronting a severe thermal crisis. Reports indicate that NVIDIA’s next-generation Blackwell AI chips are experiencing overheating when deployed in 72-GPU racks, jeopardizing timelines for AI-focused infrastructure projects. These issues highlight the inadequacy of conventional air-cooling methods and underscore the urgency for thermally-aware chip and system design. The growing power density of modern chips demands a radical rethink in design and cooling strategies. From Microsoft’s development of liquid-cooled servers with custom security

and storage chips to Alibaba Cloud’s service disruptions due to heat-induced failures at Digital Realty’s SIN11 facility, the need for resilient and thermally balanced architectures has become critical. Past incidents like the OVHcloud fire in France and recent concerns about chip aging and supply chain shortages further demonstrate the fragility of existing data center infrastructure under thermal and operational stress. At the heart of these challenges lies the structural inefficiency of 2D monolithic ICs. As Moore’s Law slows, the continued miniaturization of transistors has hit physical walls, aggravating thermal density, limiting interconnect scalability, and diminishing overall reliability. With long wirelengths, poor heat spreading, and constrained layout freedom, 2D architectures struggle to meet the demands of heterogeneous and multi-die systems.

To address these challenges, 3D Integrated Circuits (3D-ICs) [1] offer a compelling architectural paradigm shift. By vertically stacking dies, 3D-ICs enable tighter integration of disparate components—logic, memory, analog and significantly reduce interconnect latency and footprint. Technologies such as Through-Silicon Via (TSV) stacking, Monolithic 3D Integration (M3D), and Face-to-Face (F2F) bonding provide varying trade-offs in vertical bandwidth, integration density, and fabrication complexity. While TSVs offer mature support for logic-memory integration, they incur parasitics and congestion. M3D unlocks transistor-level granularity but faces yield and process maturity challenges. F2F bonding strikes a practical balance, enabling chiplet-based architectures with high vertical I/O density and reduced routing overhead. Despite these advantages, 3D integration introduces formidable floorplanning complexities: thermal coupling between layers, vertical congestion, TSV placement trade-offs and spatial imbalance across stacked dies. Conventional Electronic Design Automation (EDA) workflows treat thermal, placement and interconnect stages in isolation, resulting in slow convergence and suboptimal outcomes.

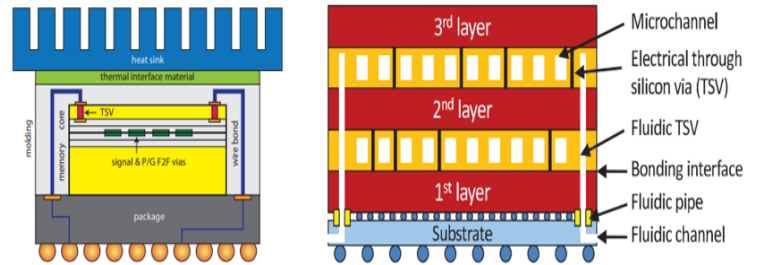


Fig. 1: 3D Chiplets Floorplanning Methods

To bridge this gap, we present a unified floorplanning framework that brings together Multi-Agent Reinforcement Learning (MARL), RC-based thermal modeling and compact liquid cooling topology integration [16]. Each die in the 3D stack is assigned an intelligent MARL agent that performs localized macro placement under thermal, power and congestion constraints. A centralized vertical coordination agent operates across tiers, jointly optimizing TSV insertion, inter-layer communication and microchannel pipe placement. Thermal features are dynamically derived from compact models (e.g., PACT) that capture localized heat transfer, fluid flow and power dissipation. The learning process is guided not only by wirelength and area metrics but also by live thermal gradients and coolant flow proximity. This inter-agent coordination ensures that decisions made at one layer consider their global impact, thereby enhancing overall floorplan efficiency and mitigating thermal hotspots.

To accelerate both training and deployment, we introduce a **hardware-mapped MARL system**, where each policy network is deployed on an *Agent Processing Unit (APU)*. Every APU features an **Intelligent Experience Relay Buffer (IERB)**, which mimics cache replacement logic to prioritize experience replay based on *TD-error and policy entropy*, thereby maximizing information gain per iteration. A private L1 buffer handles local transitions, while a shared L2 cache enables low-latency inter-agent synchronization via a *NoC-based interconnect*, emulating a decentralized training kernel. This hardware-software co-design allows near real-time execution of multi-agent learning over stacked chiplets—transforming what was traditionally an offline, compute-heavy process into a scalable, on-chip inference and optimization fabric.

- We present **THERMAL-MARL**, the first *two-phase cyclic Multi-Agent Reinforcement Learning framework* for 3D chiplet floorplanning, combining intra-layer policy networks with a cross-layer coordination agent for thermally balanced macro placement across dies.
- **Joint Macro, TSV, and Liquid Cooling Pipe Optimization:** We introduce a vertically coordinated learning agent that simultaneously manages macro reassignment, TSV insertion, and microchannel liquid cooling pipe routing, enabling unified co-optimization of inter-layer thermal hotspots and routing congestion.
- **Physics-Guided Thermal Modeling with Flow-Aware Placement:** We integrate compact RC-based thermal modeling (via PACT [18]) directly into the learning loop, enabling agents to receive real-time temperature and fluidic feedback. Leveraging this, we propose a novel flow-aware placement cost function that combines local power density, coolant flow rate, and spatial proximity to cooling inlets—ensuring thermally informed macro placement and effective hotspot mitigation.
- **Technology-Aware State Representation:** Each agent receives layer-specific parameters such as thermal conductivity, delay, and power characteristics, enabling heterogeneity-aware optimization tailored to each die’s

physical properties.

- **Hardware-Mapped MARL Acceleration:** We propose a many-core Agent Processing Unit (APU) architecture, where each MARL agent is mapped to a dedicated core equipped with prioritized replay buffers guided by TD-error and entropy-based sampling. Compared to traditional GPU-based training, this hardware mapping significantly reduces training latency and supports scalable, real-time floorplanning across multi-die 3D systems.

Extensive experiments demonstrate that **THERMAL-MARL** achieves superior thermal uniformity, wirelength reduction, and placement feasibility, outperforming prior 2D and 2.5D RL-based floorplanning approaches.

II. RELATED WORK AND MOTIVATION

A. Prior Efforts in Thermal-Aware 3D Floorplanning

Early 3D IC floorplanning methods predominantly relied on classical heuristics such as Simulated Annealing (SA) and Genetic Algorithms (GA), with limited scalability and oversimplified thermal assumptions. Frantz et al. [2] and Chen and Yoshimura [3] explored GA and SA-based frameworks using sequence pairs for fixed-outline placement. Hybrid strategies like ant colony + SA [4], multi-phase pipelines [5–7], and decoupled vertical–horizontal optimizations [6] were also proposed. Despite their algorithmic diversity, these approaches falter on large, thermally heterogeneous SoCs due to rigid representations, long runtimes, and lack of coupling between physical and thermal dynamics. Recent AI-driven frameworks, led by Google’s AlphaChip [8], have demonstrated success in 2D macro placement using deep reinforcement learning with graph embeddings. Other works leveraged Q-learning [9], GraphSAGE with actor–critic models [10], and hypergraph-based policy networks [11] to enhance placement efficiency. However, all these methods operate in flat 2D domains and neglect vertical heat propagation, TSV congestion, and cooling system integration—critical factors in stacked-die systems.

B. Motivation for a Learning-Driven Framework

Despite progress in RL-based placement, no existing method supports *dynamic adaptation* to the multi-dimensional complexity of modern 3D ICs. Current frameworks assume thermal homogeneity and static architecture layers, failing to scale across varying technology nodes and power densities. Key limitations include:

- **Thermal myopia:** Prior methods lack physically grounded thermal models—ignoring inter-die heat coupling and real-time power dissipation feedback.
- **Disconnected optimization:** TSV and cooling placement are treated as afterthoughts or omitted entirely, missing their interplay with macro placement.
- **No cross-tier adaptability:** Agents are blind to tier-specific technology, thermal conductivity, and performance/power asymmetry across stacked dies.

Our Solution: We introduce a unified MARL-based 3D floorplanning framework that co-optimizes macro layout, TSV

routing, and microchannel cooling—all inside the learning loop. Guided by compact RC-based thermal models and coolant-aware state features, our agents make tier-aware, physically-informed placement decisions. This sets a new foundation for thermally resilient, scalable chiplet systems under realistic constraints.

III. PROPOSED METHODOLOGY

We propose a two-phase cyclic multi agent reinforcement learning framework for 3D integrated chiplets floorplanning, involving coordinated interactions between horizontal (intra-layer) agents and a vertical (inter-layer) thermal-aware agent. The algorithm proceeds in multiple cycles. In each cycle, horizontal agents perform local floorplanning actions within their assigned layers. After several such steps, a vertical agent globally evaluates thermal and placement conditions and adjusts macro distributions across layers. This process continues iteratively until convergence, defined by a stability threshold in the cost function.

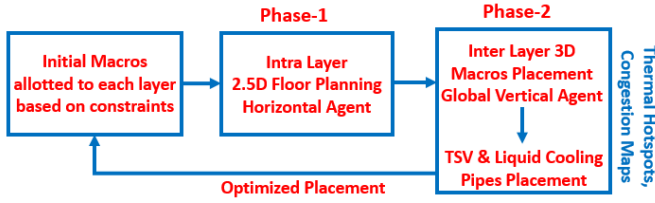


Fig. 2: Proposed Multi Cyclic - 2 Phase 3D Floor planning

A. Phase I: Intra-layer 2.5D Floorplanning

In the first phase, each physical die layer $l \in \{1, 2, \dots, L\}$ is assigned a dedicated reinforcement learning agent A_l , which coordinates horizontal macro placement within its respective layer. Given a set $M = \{m_i \mid i \in \mathbb{Z}^+, 1 \leq i \leq N_m\}$ of rectangular macros, where m_i denotes the i -th macro and N_m is the total number of macros, each macro is defined by its width w_i , height h_i , area a_i and power p_i . The environment provides each agent with a layer-specific observation space s_t^l at time step t , comprising relevant physical, thermal and connectivity features to guide placement decisions under fixed outline constraints.

Matrix-Based State Space for Agent A_l	
State Component	Description
Macro Occupancy Map	Grid $(H \times W)$ with macro IDs in occupied cells, 0 elsewhere.
Thermal Map [14]	Real-time temperature distribution matrix for layer l .
Congestion Map $C^l(x, y, t)$	Routing congestion matrix, updated after placement changes.
Liquid Cooling distance $L^l(x, y)$	Binary grid indicating presence of liquid cooling paths.
Power Density Map $P^l(x, y)$	Power consumption per cell based on macro mapped to (x, y) .
Wire length $N^l(x, y)$	Weighted connectivity per grid cell to capture wirelength gradient.
Fixed Boundary Constraints	Grid with legal placement zones (1 = valid, 0 = blocked).

Floorplan Constraints and Routing Congestion: The position of a macro is denoted by its lower-left coordinates (x_i, y_i) in the 2D plane [15] and its die layer index z_i , where $1 \leq z_i \leq K$ and K is the total number of tiers. Macros are categorized as either hard (GPU cores) or soft (stacked memory). To maintain design feasibility, hard macros are restricted to the lower tiers due to power delivery and thermal constraints, while soft macros are permitted to be placed in upper layers. To evaluate routing feasibility and resource utilization, the 2D layout of each device layer is partitioned into a regular grid of square regions as depicted in Fig. 3. The **congestion** in a particular box is computed as Eqn. 1.

$$\text{Congestion}_{(i,j)} = \frac{\text{Total routing demand in box } (i,j)}{\text{Routing capacity of box } (i,j)} \quad (1)$$

States with higher congestion values indicate potential routing hotspots, which can result in increased interconnect delays or routing failures. This discourages the RL agent from placing macros in such regions. Minimizing congestion across the layout is essential for ensuring design feasibility, as well as enhancing thermal performance and overall reliability.

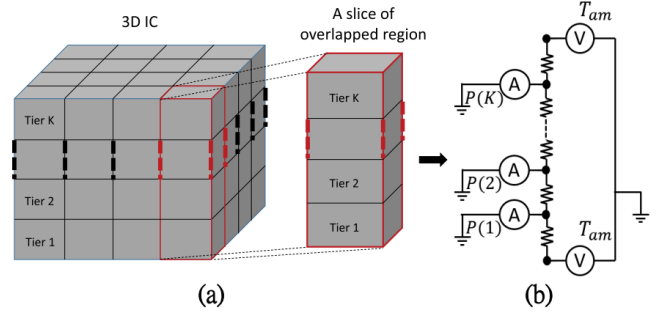


Fig. 3: Grid Constrained Floor planning

PACT-based Thermal Modeling for MARL State: To provide accurate temperature feedback to MARL agents during 3D floorplanning, we incorporate a compact RC-network-based thermal model as implemented by the PACT simulator [18]. PACT models temperature by constructing a thermal RC netlist, where heat conduction paths are represented as resistors and thermal storage is captured through capacitors. Power dissipation in chiplets is treated as current sources, and the resulting temperature at each grid point corresponds to the voltage obtained after simulation. The final temperature distribution is derived by solving the Eqn. 2.

$$C \cdot \frac{dT}{dt} + G \cdot T = P \quad (2)$$

where C is the thermal capacitance matrix, G is the thermal conductance matrix, T is the temperature vector to be solved and P is the power dissipation vector. In steady-state simulations, the Eqn. 2 reduces to Eqn. 3.

$$G \cdot T = P \Rightarrow T = G^{-1} \cdot P \quad (3)$$

PACT executes this through SPICE-based simulation, where thermal resistors, capacitors and current sources are directly mapped to circuit components. This temperature output is converted into grid-based thermal features and passed to each layer-specific RL agent for thermally informed placement decisions.

Wirelength Estimation: To evaluate the circuit wirelength during training and inference, we adopt the Half-Perimeter Wirelength [15]. HPWL is a widely used approximation method for estimating total wirelength in placement problems. The total HPWL is computed in Eqn. 4. where $x_{\max}$ and $x_{\min}$ are the maximum and minimum x-coordinates of pins in net j and similarly for $y_{\max}$ and $y_{\min}$.

$$\text{HPWL} = \sum_{\text{net}_j} [(x_{\max} - x_{\min}) + (y_{\max} - y_{\min})] \quad (4)$$

Liquid Cooling and Placement Metric Justification: To improve vertical heat dissipation in 3D ICs, we incorporate interlayer microchannel liquid cooling within the thermal modeling framework of PACT. As shown in Fig. 4, a fluidic cooling layer is inserted between silicon tiers. Each grid node within this layer is modeled using directional thermal resistors and convection-aware voltage-controlled current sources. The effective heat transfer coefficient along both vertical and side walls of the microchannel is expressed in Eqn. 5, where k_{coolant} denotes the coolant’s thermal conductivity, Nu is the Nusselt number, and d_h is the channel’s hydraulic diameter.

$$hf_{\text{vertical}} = hf_{\text{side}} = \frac{k_{\text{coolant}} \cdot \text{Nu}}{d_h} \quad (5)$$

$$J_{\text{conv}} = c_{\text{conv}} \cdot (T_{\text{in}} - T_{\text{out}}) \quad (6)$$

The convective behavior of the fluid is modeled using Eqn. 6, where c_{conv} is the liquid convection coefficient and T_{in} , T_{out} represent the inlet and outlet temperatures of adjacent fluid nodes. These additions enable PACT to simulate the thermal impact of microchannel cooling under various flow conditions. To complement this cooling architecture, we integrate a thermal-aware placement strategy that accounts for local power density and coolant availability. While classical strategies like Manhattan-distance-based spreading reduce proximity-induced hotspots but they often ignore dynamic thermal effects and flow variability. Therefore, we introduce a flow-sensitive thermal placement metric:

$$\text{Thermal_Cost}_i = \frac{P_i}{F_i} \cdot D(i, \text{CoolantSource}) \quad (7)$$

Here, P_i is the power dissipated by unit i , F_i is the coolant flow rate at that location, and $D(i, \text{CoolantSource})$ is the distance to the nearest coolant inlet. This cost function encourages placement of high-power units near high-flow regions, thereby reducing peak temperature and improving thermal reliability. The metric can be extended to include

interlayer thermal resistances, capturing vertical heat spread across the stack for holistic thermal optimization.

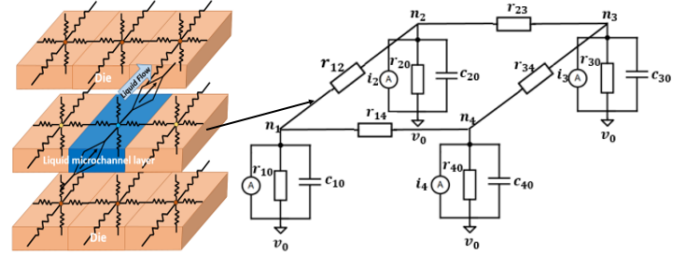


Fig. 4: Liquid cooling Modeling for feature extraction

Intra-Layer Floorplanning Action Space: For each die layer, the policy network selects from four discrete actions to optimize floorplanning shown in table below. These actions are sampled from the policy distribution $\pi_{\theta}(a|s)$ produced by the agent. This setup enables efficient policy updates using PPO while preserving placement feasibility and thermal-awareness at the layer level.

Intra-Layer Action Space for Policy Network	
Action Name	Description
Swap_One_Axis	Swap a pair of blocks along either the X_t or Y_t sequence to introduce localized perturbation.
Swap_Both_Axes	Simultaneously swap a block pair in both X_t and Y_t to induce stronger local reordering.
Reposition_Block	Move a selected block to a new position within sequence to allow flexible spatial rearrangement.
Rotate_Block	Rotate a selected block by 90° to explore orientation-dependent effects on wirelength.

Cumulative Reward: Eqn. 8 is the reward function for the horizontal agent, designed to capture the global impact of local placement decisions across thermal, routing, and spatial objectives. This reward formulation enables the policy network to learn a balanced placement strategy that minimizes peak temperature, wiring overhead, and routing congestion while also promoting thermally-aware macro positioning. Each term contributes to system-wide optimization and ensures that learning remains sensitive to physical design constraints during intra-layer floorplanning.

$$r_t = -\lambda_1 \cdot \max_{(x,y)} T(x, y, t) - \lambda_2 \cdot \text{HPWL}_t - \lambda_3 \cdot \sum_{(i,j)} \text{Congestion}_{(i,j)}^t - \lambda_4 \cdot \sum_i \text{ThermalCost}_i^t \quad (8)$$

B. Phase II: Inter-layer Thermal-Aware Macro Placement

After k horizontal iterations, the vertical agent A^{vert} becomes active. It aggregates thermal and power vectors from all L layers into a unified global stack state, given in Eqn. 9:

$$S_t^{\text{global}} = \bigcup_{l=1}^L \{T^l(x, y, t), P^l(x, y), (x_i^l, y_i^l)\} \quad (9)$$

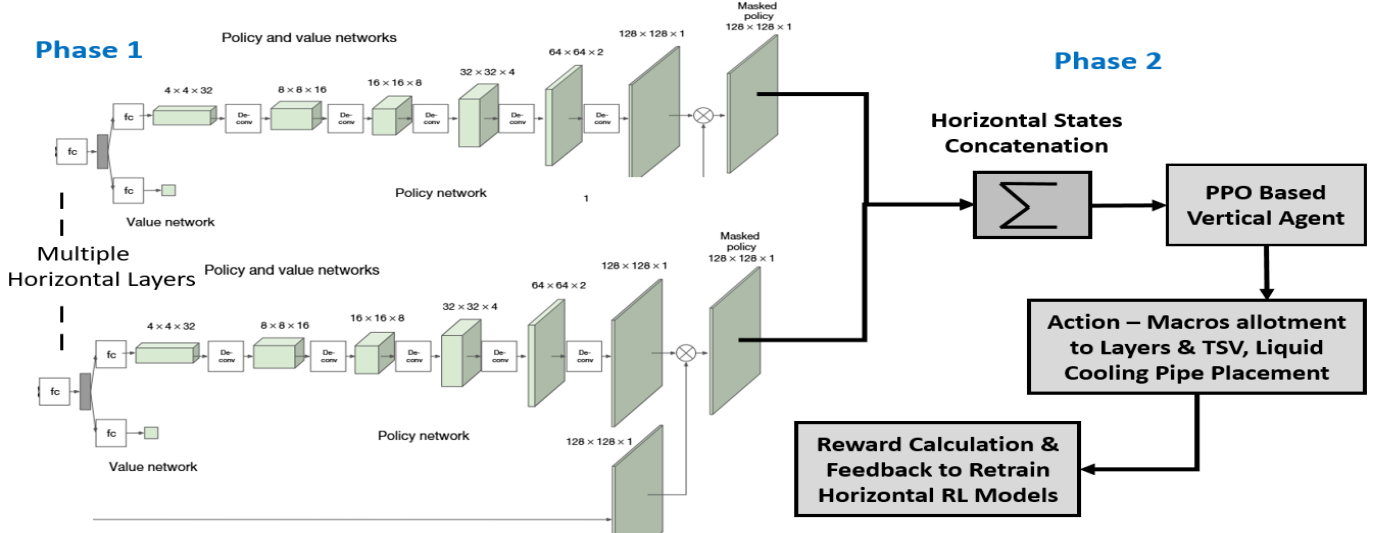


Fig. 5: Proposed Neural MARL Architecture

This state captures the spatial placement, power distribution, and temperature maps of the entire 3D stack. The vertical agent evaluates S_t^{global} to detect inter-layer thermal hotspots, unbalanced power densities and vertical congestion. Based on this, it generates macro migration actions defined in Eqn. (10):

$$a_t^{\text{vert}} = \{m_i : l \rightarrow l' \mid T^l(x_i^l, y_i^l, t) > \tau_T \wedge P^l(x_i^l, y_i^l) > \tau_P\} \quad (10)$$

Here, τ_T and τ_P are thermal and power thresholds and l' is the target layer selected to reduce cumulative violations.

Joint TSV and Liquid Pipe Placement: Once macro migration decisions are made, the vertical agent co-optimizes TSV and liquid pipe placement. Placement cost function is modeled

$$\text{Cost}(x, y, l) = \alpha \cdot \text{Congestion}(x, y, l) + \beta \cdot T(x, y, l) \quad (11)$$

in Eqn 11. TSVs are placed in cooler, less congested zones to support interconnect density, while liquid cooling pipes are aligned to high-thermal areas to dissipate heat efficiently. Scalar reward function for vertical agent is shown in Eqn. 12, which reflects its effectiveness in reducing system-wide thermal peaks and vertical congestion.

$$r_t^{\text{vert}} = -\max_{(x,y)} T(x, y, z) - \lambda_2 \cdot \text{VD}_t \quad (12)$$

Here, VD_t is vertical density representing interconnect pressure due to TSV usage. This reward encourages better thermal and 3D distribution. Transition to Next Phase: Once macro, TSV and cooling pipe placements are finalized, the updated stack state is sent to all horizontal agents. Each layer reinitializes with new configuration and resumes localized optimization, continuing the hierarchical loop until convergence.

IV. PROPOSED MARL NEURAL ARCHITECTURE

In the proposed MARL framework for 3D IC floorplanning, each agent operates using a structured neural architecture as shown in Fig. 5. The environment is a multi-tier 3D space,

where agents sequentially place computational elements such as logic blocks, memory macros, or accelerators into valid 3D coordinates (x, y, z) . Each agent perceives its surroundings through a graph-based state and acts using a learned policy.

A. Graph-Based State Representation & GNN layers

As shown in Fig. 5, each die layer receives its own environmental state in the form of matrix-based features such as thermal maps, congestion levels, and tier utilization. These are encoded into a dynamic graph $G = (V, E)$, where each node $v \in V$ represents a macro and edges $e \in E$ encode spatial and thermal interdependencies including TSVs. Each MARL agent processes this graph via a stack of Graph Neural Network (GNN) layers. Node embeddings are iteratively updated using neighbor aggregation modulated by edge attributes, enabling agents to capture spatial feasibility and thermal context. Typically 3 to 5 GNN layers are used, producing a contextual node embedding h_v . The embeddings are passed into two MLP heads: the *policy head* outputs a probability distribution over all valid 3D coordinates (x, y, z) ; the *value head* estimates the long-term reward. These guide the agent's next placement action, executed within its tier. All horizontal agents operate in parallel, independently making local placement decisions. Their embeddings and policy outputs are sent to a vertical coordination agent for inter-tier reasoning. The policy π_θ is optimized using Proximal Policy Optimization (PPO), with the clipped objective function given in Eqn. 13.

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (13)$$

The vertical agent uses pooling or attention over tier-wise embeddings to generate coordination signals for thermal balancing, macro redistribution, and TSV routing. These signals are fed back to horizontal agents to refine their policies. In upper layers, the vertical agent also promotes soft macro placement to reduce area footprint.

B. Training Procedure

Training occurs in two phases. First, supervised pretraining is conducted using known floorplans to accelerate convergence. The training dataset consists of (state, placement) pairs, and the loss is the mean squared error between predicted and ground-truth rewards. Subsequently, MARL-based fine-tuning is performed, optimizing placement via rewards based on wirelength, thermal profile, TSV usage and congestion. The agent's value function $V(s_t)$ is trained by minimizing:

$$\mathcal{L}_{\text{value}} = (V(s_t) - R_t)^2 \quad (14)$$

This hybrid of supervised and reinforcement learning enables fast and thermally-aware macro placement across 3D tiers.

V. PROPOSED HARDWARE ARCHITECTURE FOR RL-DRIVEN 3D CHIPLET FLOORPLANNING

Implementing Multi-Agent Reinforcement Learning (MARL) for 3D chiplet floorplanning requires efficient inter-agent communication, prioritized experience replay and scalable hardware infrastructure. Conventional GPU-based training [12] is often limited by bandwidth and memory reuse inefficiencies, slowing down policy convergence and evaluation throughput. Being motivated from [14], we propose a dedicated system-level MARL accelerator (Fig. 6) where each RL agent is mapped to a specialized Agent Processing Unit. The architecture is purpose-built for 3D chiplet layout optimization, enabling simultaneous training and inference across die layers. Key components of Accelerator are:

- **APU Core:** Each APU integrates a CNN accelerator with a three-tier memory system: (i) a PE-local buffer for operand reuse, (ii) global memory for intra-agent state sharing, and (iii) input/output cache buffers for streaming minibatch data. A sparsity-aware compute engine skips zero-valued operations, reducing energy and latency. Post-processing is offloaded to a lightweight unit that performs activation and normalization tasks without CPU intervention.
- **Intelligent Experience Relay Buffer (IERB):** Acting as a local L1 neural cache, the IERB filters and retains transitions based on a hybrid score combining Temporal Difference (TD) error and policy uncertainty (e.g., entropy or advantage variance). Critical samples are prioritized for reuse, while lower-impact experiences are offloaded to a shared buffer.
- **Shared L2 Cache and NoC Interconnect:** Experiences offloaded from local buffers are stored in a centralized L2 cache accessible by all APUs. A high-bandwidth Network-on-Chip (NoC) ensures fast experience sharing, enabling dynamic coordination across agents for thermally balanced and congestion-aware floorplanning decisions.

Each agent interacts with its local environment, while the memory and interconnect hierarchy enables global convergence through hybrid memory reuse and distributed experience relay.

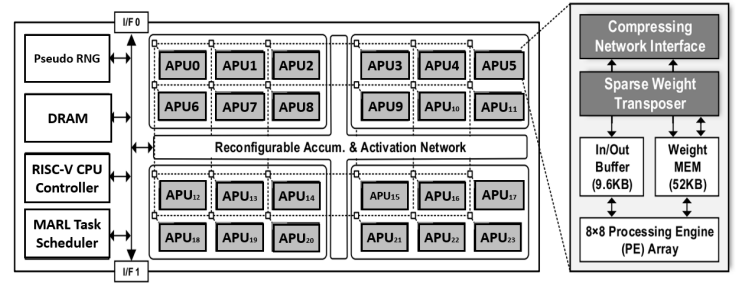


Fig. 6: Proposed Many-Core MARL Accelerator Architecture

A. Agent Processing Unit - CNN Accelerator

Each APU includes a sparse-aware CNN accelerator optimized for inference in reinforcement learning workflows. Fig. 6 illustrates the internal components:

- **Compression Network Interface:** Decompresses incoming sparse models by using , maintaining efficiency in memory access and minimizing bandwidth usage.
- **Sparse Weight Transposer:** Reorganizes model weights to optimize broadcasting across PEs, aligning with tile-based parallel processing.
- **Input/Output Buffers:** Temporarily store input feature maps and output activations, facilitating seamless data movement within the APU.
- **8x8 PE Array:** The compute core comprises 64 processing elements that execute parallel multiply-accumulate operations, tailored for CNN-based policy and value function inference.

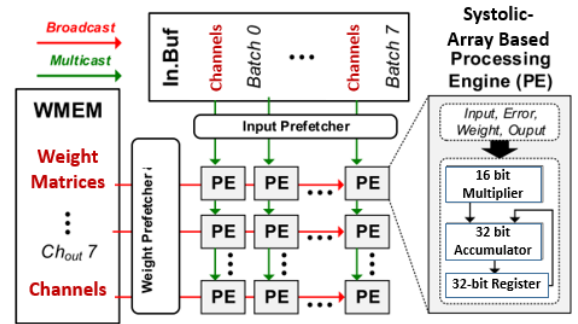


Fig. 7: Internal APU Core Microarchitecture

Each PE employs a 16-bit floating point (FP16) unit for computation, balancing numerical precision with energy efficiency. Preloading of weights and activations into local buffers ensures low-latency execution with minimal memory stalls. The architecture is optimized for low-power deployment, supporting real-time inference for MARL agents in embedded and edge-AI platforms. Overall, this accelerator design enables scalable, parallelizable and energy-aware deployment of reinforcement learning driven optimized 3D chiplet floorplanning.

B. Proposed Memory System

To accelerate convergence and enhance thermal-aware chiplet placement, we propose a hierarchical prioritized experience memory inspired by Prioritized Experience Replay of DQN networks [13]. Each transition $e_i = (s_i, a_i, r_i, s'_i)$ is evaluated using a composite priority score that reflects its impact on learning and spatial thermal optimization. As shown in Eqn. (1), the priority score P_i is computed using the temporal-difference error $|\delta_i|$, the policy deviation $(1 - \text{PolicyAcc}_i)$ and the spatial influence SpatialImpact_i , combined with tunable weights $\lambda_1, \lambda_2, \lambda_3$:

$$P_i = \lambda_1 |\delta_i| + \lambda_2 (1 - \text{PolicyAcc}_i) + \lambda_3 \cdot \text{SpatialImpact}_i \quad (15)$$

Transitions are stored across a three-level memory hierarchy: Level-1 caches top- K percentile high-priority samples for frequent GPU access, Level-2 holds medium-priority experiences to track evolving dynamics, and Level-3 archives low-priority or aged samples to maintain diversity and prevent overfitting. Sampling from memory is guided by proportional prioritization as shown in Eqn. 16.

$$P(k) = \frac{(P_k + \epsilon)^\alpha}{\sum_j (P_j + \epsilon)^\alpha}, \quad w_k = \left(\frac{1}{N \cdot P(k)} \right)^\beta \quad (16)$$

where N is the memory size, α controls prioritization sharpness and β is annealed from β_0 to 1. The final TD update is scaled using $w_k \cdot \delta_k$, ensuring robust learning. This memory system dynamically adapts to policy evolution, emphasizes spatially and thermally critical transitions and improves training throughput for 3D chiplet placement.

VI. EXPERIMENTAL SETUP

We evaluate **THERMAL-MARL** on two heterogeneous 3D-IC systems based on contemporary AI datacenter designs. **System-A** includes dual GPUs (GPU-1 and GPU-2), multi-core CPU [17], shared GPU RAM, stacked HBM and DRAM, similar to NVIDIA's DGX series. It focuses on vertical data movement, posing challenges in thermal decoupling and TSV congestion. **System-B** couples a mesh-networked CPU, dedicated-RAM NP, stacked DRAM, IVRs, I/O controller and AI accelerator following the models of Meta's MTIA and AWS Trainium. It struggles because layers differ in their power densities, latencies and cooling requirements. Simulations utilize a small RC-based thermal model from PACT [18], expanded to include TSV thermal resistance and active liquid cooling. These are incorporated into the agent's observation-feedback loop for temperature-conscious learning. The multi-agent reinforcement learning (MARL) framework is then trained with Proximal Policy Optimization (PPO) algorithm in PyTorch on an AMD EPYC CPU system with 128 threads and NVIDIA A100 GPUs on Google Colab. The final tests are conducted on the A100 GPU and suggested custom RTL-based manycore APU mapped simulator with L1 and shared L2 caches for performance and thermal exploration.

A. Evaluation Results

THERMAL-MARL continuously outperforms Simulated Annealing (SA) and Bayesian Optimization (BO) on both systems. It attains a 5.4°C decrease in the peak temperature and an 11.7% shorter wirelength than SA, as well as 7.7% fewer TSVs and much greater cooling pipe density. The runtime is also decreased by around 18.4% with respect to SA. In System-B, it additionally enhances thermal balance with a 6.6°C reduced peak temperature and lowers runtime more than 25% than BO and also decreases the mean temperature by 3.7°C. These are due to direct incorporation of thermal feedback into policy learning and structural knowledge of TSV/coolant constraints.

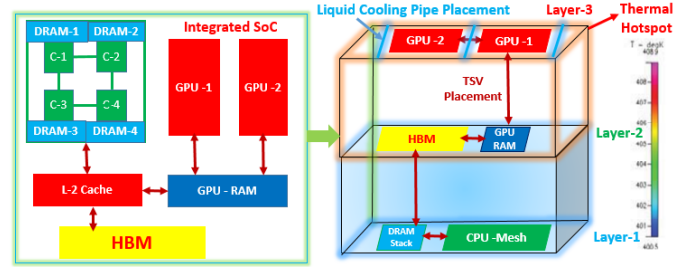


Fig. 8: System-A 3D Floor planning

The final macro placements across layers of System-A, shown in Fig. 8, balance spatial separation and thermal decoupling of high-power components. Layer 1 (Bottom – Blue) hosts the CPU Mesh and DRAM Stack, ensuring that memory and logic remain cool and functional. Layer 2 (Middle – Sky Blue) contains HBM and GPU RAM, interconnected via TSVs, serving as the communication bridge between the lower and upper layers. The top Layer 3 (Red) includes the high-power GPU-1 and GPU-2, with liquid cooling pipes positioned to manage thermal hotspots. TSVs vertically interconnect the layers, optimizing data flow between GPU ↔ RAM, RAM ↔ HBM, and HBM ↔ CPU/DRAM. The final macro

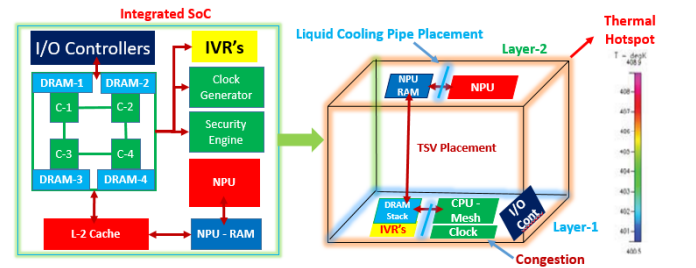


Fig. 9: System-B 3D Floor planning

placements across layers of System-B is shown in Fig. 9, combines performance-conscious and thermal-efficient design techniques to solve important issues in 3D IC integration. In Layer-1 (bottom), large functional blocks like the CPU with a mesh-based interconnect, integrated voltage regulators (IVRs), clock generator and stacked DRAM are placed strategically to reduce communication latency and power delivery efficiently.

TABLE I: Comparative Evaluation of 3D Floorplanning Methods on Two Heterogeneous Systems

System-A: Dual GPU + Cache + CPU Stack						
Method	Max Temp (°C)	Avg Temp (°C)	Wirelength (mm)	TSVs Used	Cooling Pipe Density	Runtime (s)
Simulated Annealing (SA)	91.6	76.4	101.2	912	Low	1142
Bayesian Optimization (BO)	88.9	74.5	94.8	980	Medium	1583
THERMAL-MARL (Ours)	86.2	70.8	89.3	842	High	932
System-B: CPU + NPU + DRAM + I/O + IVRs + Clock Generator						
Method	Max Temp (°C)	Avg Temp (°C)	Wirelength (mm)	TSVs Used	Cooling Pipe Density	Runtime (s)
Simulated Annealing (SA)	93.8	78.6	109.4	1015	Low	1365
Bayesian Optimization (BO)	90.4	75.8	103.2	968	Medium	1760
THERMAL-MARL (Ours)	87.2	72.1	96.7	905	High	1018

The I/O controller is collocated here so that access to memory and CPU can be provided quickly and bottlenecks of data transfer avoided. Layer-2 (top) holds the neural processing unit (NPU) and dedicated RAM, both of which tend to generate significant heat upon execution. Thermal stress is avoided through the positioning of liquid cooling pipes right above the NPU such that effective thermal dissipation can be done from the top layer. Through-silicon vias (TSVs) link the two layers, allowing for smooth vertical communication and power delivery. This enhanced configuration, with improved cooling infrastructure, mesh CPU fabric and extra memory and control units, offers a well-balanced solution for power management, thermal reliability and communication efficiency in high-performance 3D systems.

TABLE II: Results on GPU vs. Custom APU

Platform	Training Time (Epochs)	Inference Time (s/episode)	Energy per Rollout (J)
NVIDIA A100 GPU	842	0.216	7.84
Custom APU (Ours)	624	0.095	3.21

To assess architectural scalability, we also benchmark training and inference execution on both the NVIDIA A100 GPU and our custom-designed APU hardware. As shown in Table I, **THERMAL-MARL** runs faster and more energy-efficient on the APU, reducing inference time by over 55% and energy per rollout by more than 59%.

VII. CONCLUSION

We presented **THERMAL-MARL**, a reinforcement learning-based framework for thermally-aware floorplanning in heterogeneous 3D-ICs. By integrating fine-grained thermal modeling, TSV cost, and cooling feedback directly into the learning loop, our approach achieves superior placement quality over classical methods. Evaluations on realistic AI-inspired stacks demonstrate significant reductions in temperature, wirelength, and runtime. Further, our custom APU-based execution shows enhanced efficiency over GPU baselines, validating the scalability of **THERMAL-MARL** for next-generation chiplet systems.

REFERENCES

- [1] Sourì, Multiple Si layer ICs: Motivation, performance analysis, and design implications. In Proceedings of the 37th Annual Design Automation Conference, Los Angeles, CA, USA, 5–9 June 2000; pp. 213–220.
- [2] Frantz, I. 3D-IC floorplanning: Applying meta-optimization to improve performance. In Proceedings of the 2011 IEEE/IFIP 19th International Conference on VLSI and System-on-Chip, Kowloon, Hong Kong, 3–5 October 2011; IEEE: Piscataway, NJ, USA, 2011; pp. 404–409.
- [3] Chen, S.; Yoshimura, T. Multi-layer floorplanning for stacked ICs: Configuration number 2010, 43, 378–388.
- [4] Xu, Q., Combining the ant system algorithm and simulated annealing for 3D/2D fixed-outline floorplanning. *Soft Comput.* 2016, 40, 150–160.
- [5] Guler, A.; Jha, N.K. Hybrid monolithic 3-D IC floorplanner. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 2018, 26, 1868–1880.
- [6] Zhu, H.Y. Floorplanning for 3D-IC with Through-Silicon via co-design using simulated annealing. EMC/APEMC, Singapore, 14–18 May 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 550–553.
- [7] Shanthi, Thermal Aware Floorplanner for Multi-Layer ICs with Fixed-Outline Constraints. In Proceedings of the 2021 2nd International Conference on Communication, 16–17 December 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–6.
- [8] A graph placement methodology for fast chip design. Azalia Mirhoseini, 2021. *Nature*, 594(7862), pp.207–212.
- [9] He, Z.; Ma, Y.; Zhang, L.; Liao, P.; Wong, N.; Yu, B.; Wong, M.D. Learn to floorplan through acquisition of effective local search heuristics. In Proceedings of the 2020 IEEE 38th International Conference on Computer Design (ICCD), Hartford, CT, USA, 18–21 October 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 324–331.
- [10] Xu, Q.; Geng, H.; Chen, S.; Yuan, B.; Zhuo, C.; Kang, Y.; Wen, X. GoodFloorplan: Graph Convolutional Network and Reinforcement Learning-Based Floorplanning. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 2021, 41, 3492–3502.
- [11] Amini, M.; Zhang, Z.; Penmetsa, S.; Zhang, Y.; Hao, J.; Liu, W. Generalizable Floorplanner through Corner Block List Representation and Hypergraph Embedding. *ACM SIGKDD DC*, USA, 14–18 August 2022; pp. 2692–2702.
- [12] JEYA PRAKASH KADAMBARAJAN, SIVAKUMAR POTHIRAJ, and PANDIARAJ KADARKARAI, “GPU Implementation of Thermal Aware 3D IC Floorplanning”, *ijcism*, vol. 13, p. 8, Jan. 2021.
- [13] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” 2015, arXiv:1511.05952.
- [14] S. An, “A 8.81 TFLOPS/W Deep-Reinforcement-Learning Accelerator With Delta-Based Weight Sharing and Block-Mantissa Reconfigurable PE Array,” in *ITSC* vol. 71, no. 5, pp. 2529–2533, May 2024.
- [15] Murata, H.; Fujiiyoshi, K.; Nakatake, S.; Kajitani, Y. VLSI module placement based on rectangle-packing by the sequence-pair. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 1996, 15, 1518–1524.
- [16] Jang, H.B., Yoon, I., Kim, C.H., Shin, S., Chung, S.W.: The impact of liquid cooling on 3D multi-core processors. In: *IEEE International Conference on Computer Design, ICCD (2009)*
- [17] Monchiero, M., Canal, R., Gonz’alez, A.: Design space exploration for multicore architectures: a power/performance/thermal view. In: *ICS 2006: Proceedings of the 20th Annual International Conference on Supercomputing*, pp. 177–186. ACM, New York (2006)
- [18] Z. Yuan, “PACT: An Extensible Parallel Thermal Simulator for Emerging Integration and Cooling Technologies,” *ICCAD (2021)*